

UniApp 布局方案大揭秘：从原生 CSS 到第三方组件库

你想知道在 uniApp 开发中，布局是否只能用原生 CSS，以及有没有官方或第三方的便捷布局方案，这个问题很贴合实际开发需求，毕竟纯手写 CSS 布局确实效率不高。

一、uniApp 布局并非只能用原生 CSS

uniApp 基于 Vue 生态，布局方式是**多样化**的，原生 CSS 只是基础，还支持更高效的布局方案，主要分为三类：

1. 官方内置的布局能力（无依赖，优先推荐）

uniApp 本身对布局做了适配和简化，无需额外安装：

- **Flex 布局（核心）**：uniApp 官方推荐的主流布局方式，底层已对各端（小程序、App、H5）做了兼容，只需通过 `display: flex` 即可快速实现弹性布局（如居中、均分、自适应等），比传统的 float/position 更高效。
- **uni-app 内置样式类**：官方提供了一批通用布局样式类，直接给标签加 class 就能用，无需手写 CSS：

Code block

```
1  <!-- 示例：官方内置布局类 -->
2  <view class="flex justify-center align-center">居中布局</view>
3  <view class="flex justify-between">两端对齐</view>
4  <view class="w-50 h-50">宽高50%（基于父容器）</view>
5  <view class="p-20">内边距20rpx</view>
```

常用内置类：`flex` / `block` / `inline-block`、`justify-center` / `justify-between` / `align-center`、`w-xx` / `h-xx`（百分比宽高）、`p-xx` / `m-xx`（内外边距）等。

- **rpx 单位（适配核心）**：uniApp 自带的响应式单位，自动适配不同屏幕尺寸（750rpx = 屏幕宽度），无需手动写媒体查询，是布局适配的核心。

2. 官方推荐的 CSS 预处理器（提升 CSS 效率）

uniApp 原生支持 Less/Sass/Scss，可通过变量、混合（mixin）、嵌套等特性简化布局代码，比如：

Code block

```
1  <!-- 页面内使用 Scss 写布局 -->
2  <style lang="scss" scoped>
3  // 定义布局变量
4  $main-color: #007aff;
5  $gap: 20rpx;
6
7  // 嵌套布局
8  .flex-container {
9    display: flex;
10   padding: $gap;
11   .item {
12     flex: 1;
13     margin-right: $gap;
14     &:last-child {
15       margin-right: 0;
16     }
17   }
18 }
19 </style>
```

使用前只需在 HBuilderX 中安装对应的插件（如 “scss/sass 编译”）即可。

3. 成熟的第三方布局组件库（高效开发）

如果想进一步提升布局效率，可使用适配 uniApp 的第三方 UI 库，这些库内置了大量现成的布局组件，无需手写基础布局：

组件库	特点	适用场景
uView UI（推荐）	专为 uniApp 打造，布局组件丰富（栅格、弹性盒、宫格、瀑布流等），文档友好	中小项目、快速开发
uni-ui	DCloud 官方出品，轻量、适配性强，包含 grid（栅格）、flex 等布局组件	追求轻量、官方生态优先
Vant Weapp	适配小程序/uniApp，有栅格（Grid）、布局（Layout）等基础布局组件	熟悉 Vant 生态、多端适配

示例：使用 uView 的栅格布局（最常用）

1. 先安装 uView：在项目根目录执行

```
1 npm install uview-ui
```

2. 在 `main.js` 引入：

Code block

```
1 import uView from 'uview-ui';  
2 Vue.use(uView);
```

3. 页面中使用栅格布局（快速实现多列适配）：

Code block

```
1 <template>  
2   <!-- 栅格布局：一行分为4列，适配不同屏幕 -->  
3   <u-grid column="4" border="false">  
4     <u-grid-item>  
5       <view class="item">菜单1</view>  
6     </u-grid-item>  
7     <u-grid-item>  
8       <view class="item">菜单2</view>  
9     </u-grid-item>  
10    <u-grid-item>  
11      <view class="item">菜单3</view>  
12    </u-grid-item>  
13    <u-grid-item>  
14      <view class="item">菜单4</view>  
15    </u-grid-item>  
16  </u-grid>  
17  
18  <!-- 弹性布局：快速实现均分 -->  
19  <u-flex>  
20    <u-flex-item>  
21      <view class="flex-item">占1份</view>  
22    </u-flex-item>  
23    <u-flex-item>  
24      <view class="flex-item">占1份</view>  
25    </u-flex-item>  
26    <u-flex-item flex="2">  
27      <view class="flex-item">占2份</view>  
28    </u-flex-item>  
29  </u-flex>  
30 </template>
```

无需手写任何 flex 样式，直接通过组件属性（如 `column`、`flex`）即可控制布局，且自动适配多端。

二、布局方案选择建议

1. 简单布局：优先用 **uniApp 内置样式类 + Flex + rpx**，无依赖、效率高；
2. 复杂样式：搭配 **Less/Sass** 简化 CSS 编写；
3. 快速开发/复杂布局：使用 **uView/uni-ui** 的预制布局组件（栅格、弹性盒、瀑布流等），避免重复造轮子。

总结

1. uniApp 布局**不局限于原生 CSS**，官方内置了 flex 适配、rpx 单位、通用样式类，可满足基础布局需求；
2. 提升效率可使用 Less/Sass 预处理器，或第三方 UI 库（uView 最适配、uni-ui 最轻量化）；
3. 核心推荐组合：**Flex 布局 + rpx 单位 + uView 栅格组件**，兼顾适配性和开发效率。

（注：文档部分内容可能由 AI 生成）